

Multiplicadores de Lagrange aplicados a la Teoría del Consumidor

Optimización de presupuesto y diagramas de flujo del programa en C++

Angel Manuel Ruvalcaba Garcia

23 de septiembre de 2026

1. ¿Qué es un multiplicador de Lagrange?

Un multiplicador de Lagrange (λ) es una variable auxiliar que se introduce para resolver problemas de **optimización con restricciones**: maximizar (o minimizar) una función sujeta a que las variables cumplan una condición adicional (una igualdad).

En vez de resolver el problema restringido directamente, se construye una nueva función –el **Lagrangiano**– que combina la función objetivo y la restricción:

$$L(x_1, \dots, x_n, \lambda) = f(x_1, \dots, x_n) + \lambda(c - g(x_1, \dots, x_n))$$

donde f es la función que se quiere optimizar y $g(x_1, \dots, x_n) = c$ es la restricción. El multiplicador λ mide la **sensibilidad del óptimo ante un cambio marginal en la restricción**: cuánto mejora (o empeora) el valor óptimo de f si la restricción se relaja o se aprieta ligeramente.

1.1. ¿Para qué se utiliza?

Se utiliza siempre que existan dos elementos: algo que se quiere maximizar o minimizar, y un límite (recurso escaso) que no se puede exceder. Ejemplos típicos:

- Un consumidor que maximiza su utilidad sujeto a un presupuesto limitado (nuestro caso).
- Una empresa que minimiza costos de producción sujeta a producir cierta cantidad.
- Un ingeniero que maximiza la resistencia de una estructura sujeta a un límite de material.

El método convierte un problema restringido en un sistema de ecuaciones (derivadas parciales igualadas a cero) que se puede resolver con álgebra estándar.

2. Aplicación a la teoría del consumidor

2.1. Planteamiento del problema

Un consumidor quiere maximizar su utilidad $U(x_1, \dots, x_n)$ al consumir n bienes, sujeto a no exceder su ingreso disponible I :

$$\text{Maximizar: } U(x_1, \dots, x_n) \quad \text{Sujeto a: } \sum_{i=1}^n p_i x_i = I$$

donde x_i es la cantidad consumida del bien i , y p_i su precio.

2.2. El Lagrangiano

$$L(x_1, \dots, x_n, \lambda) = U(x_1, \dots, x_n) + \lambda \left(I - \sum_{i=1}^n p_i x_i \right)$$

2.3. Condiciones de primer orden

$$\frac{\partial L}{\partial x_i} = \frac{\partial U}{\partial x_i} - \lambda p_i = 0 \quad \text{para cada } i \quad \frac{\partial L}{\partial \lambda} = I - \sum_i p_i x_i = 0$$

De aquí se obtiene la condición de óptimo del consumidor:

$$\frac{\partial U / \partial x_i}{p_i} = \lambda \quad \text{para todo } i$$

Es decir, en el óptimo la utilidad marginal por peso gastado debe ser igual en todos los bienes. λ representa la utilidad marginal del ingreso: cuánta utilidad adicional se obtendría con un peso más de presupuesto.

2.4. Caso Cobb-Douglas: solución cerrada

Para que el programa en C++ pueda calcular el óptimo sin un método numérico iterativo, se modela la utilidad como Cobb-Douglas generalizada:

$$U(x_1, \dots, x_n) = \prod_{i=1}^n x_i^{a_i}, \quad a_i > 0$$

donde a_i representa el peso o preferencia relativa del bien i . Resolviendo el sistema de condiciones de primer orden se llega a la fórmula cerrada que implementa el programa:

$$x_i^* = \frac{a_i}{\sum_{j=1}^n a_j} \cdot \frac{I}{p_i}$$

Esto significa que el gasto óptimo en cada bien es una fracción fija del ingreso, proporcional a su preferencia relativa. El multiplicador se recupera con:

$$\lambda = \frac{a_i \cdot U^*}{x_i^* \cdot p_i}$$

usando cualquier bien i (el resultado debe ser el mismo, lo que sirve como verificación del cálculo).

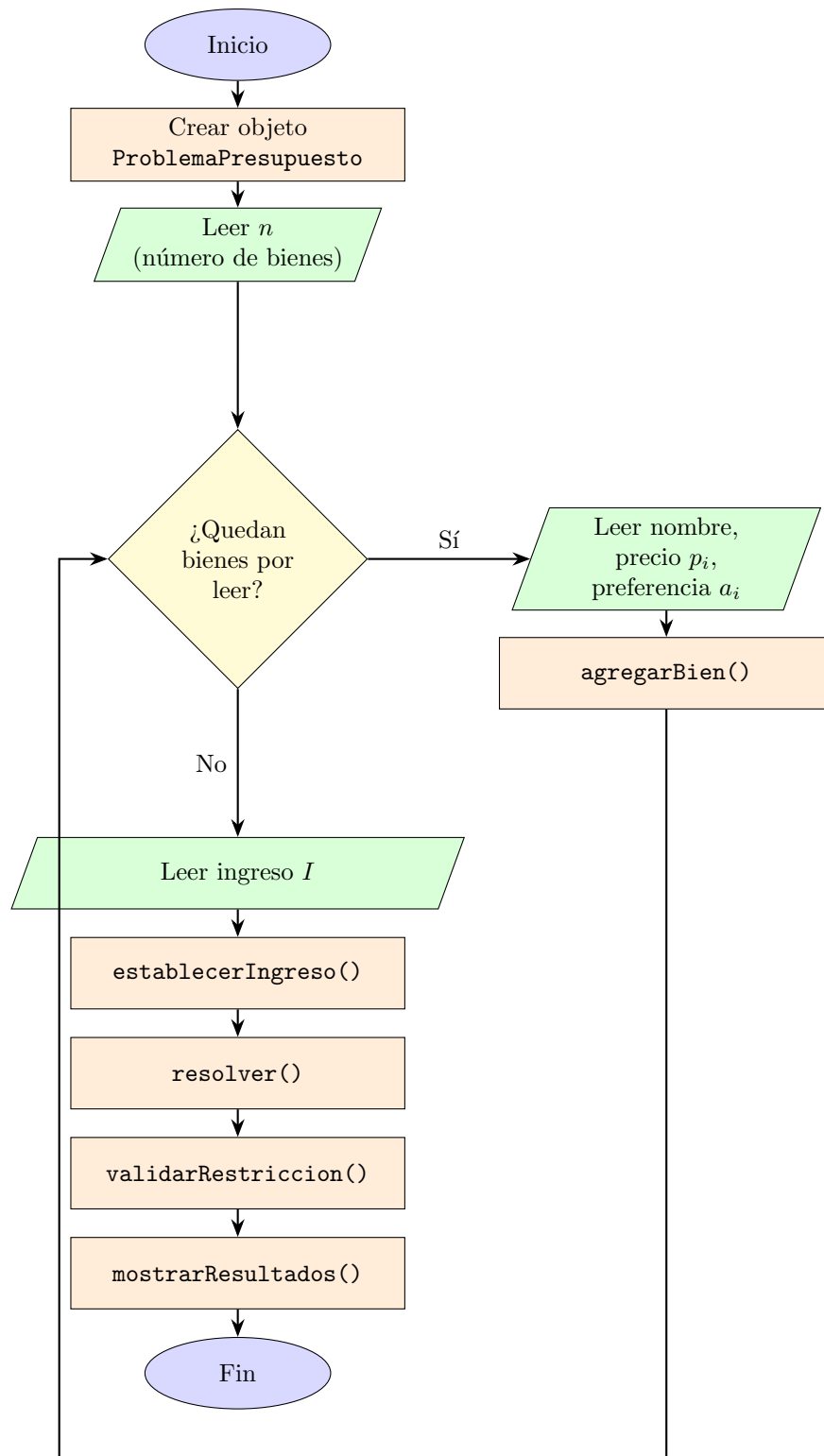
3. Estructuras de datos del programa

```
struct Bien {
    string nombre;
    double precio;           // p_i
    double preferencia;      // a_i
    double cantidadOptima;   // x_i*
    double gastoOptimo;      // p_i * x_i*
};

class ProblemaPresupuesto {
    vector<Bien> bienes;
    double ingreso;          // I
    double sumaPreferencias; // suma de a_j
    double lambda;
    double utilidadOptima;

    void agregarBien(nombre, precio, preferencia);
    void establecerIngreso(I);
    void resolver();
    double calcularUtilidad();
    double calcularLambda();
    bool validarRestriccion();
    void mostrarResultados();
};
```

4. Diagrama de flujo: main()



5. Diagrama de flujo: resolver()

